ETL-SQL Language Reference

The ETL-SQL engine is designed to parse and execute data transformations dynamically using standard SQL syntax as well as extended automation keywords.

Connection Management

Connectors define how the engine interacts with external data sources.

`CREATE CONNECTION` Defines a reusable connection to a source or destination platform. Connections must be defined before selecting from or inserting to them, unless using ad-hoc temporary tables (e.g., `#temp`).

Syntax:

```
CREATE CONNECTION <name> ON <provider>('<connection_string>') [WITH(<options>)];
```

Connection Types & Options

FLATFILE (or CSV, FILE)

For delimited text files.

- **HEADER**: `ON`, `OFF` (Default: `ON`).
- **DELIMITER**: `COMMA`, `PIPE`, `TAB`, `SEMICOLON`, `COLON`, `TILDE` or a literal `<char>`.
- **ROW_DELIMITER**: `LF`, `CR`, `CRLF` (Default: `CRLF`).
- **ENCODING**: `UTF8`, `ANSI`, `UNICODE`, `LATIN1` (Default: `UTF8`).
- **TEXT_QUALIFIER**: `DOUBLEQUOTE`, `SINGLEQUOTE`.
- **START_AT**: Line number to start reading (1-based).
- **END_AT**: Line number to stop reading.
- **COUNT_AT_END**: `ON` (Validates row count at trailer).

MSSQL (or SQLSERVER)

For Microsoft SQL Server.

- **TABLE**: Default table context.
- Supports integrated security and T-SQL pushdown.

POSTGRES (or NPSQL)

For PostgreSQL.

- **TABLE**: Default table context.
- Supports native SQL pushdown.

ORACLE

For Oracle Database.

- **TABLE**: Default table context (e.g. `SCHEMA.TABLE`).
- Supports PL/SQL pushdown.

AZURE_BLOB (or BLOB)

For Azure Blob Storage.

- **CONTAINER**: The target blob container name.
- **ACCOUNT_NAME** / **ACCOUNT_KEY**: Storage credentials.
- Supports `GET_FILE`, `PUT_FILE`, and `REMOTE_FILE_LIST`.

SMTP (or EMAIL)

For sending emails.

- **HOST**: SMTP server name.
- **PORT**: SMTP server port.
- **USERNAME** / **PASSWORD**: Authentication details.
- **USE_SSL**: `TRUE` / `FALSE`.
- **DEFAULT_FROM**: Default sender email.

PARQUET

For Apache Parquet columnar files.

- **COMPRESSION**: `SNAPPY`, `GZIP`, `LZ4`, `ZSTD` (Default: `SNAPPY`).

AVRO

For Apache Avro files.

- **SCHEMA_FILE**: Path to a `.avsc` schema file.

EXCEL

For Excel file formats (.xlsx, .xls, .xlsb).

- **SHEET**: Specific sheet name.
- **RANGE**: Explicit cell range (e.g. `'A1:B10'`).

JSON / XML

For structured document files.

- **ROOT_PATH**: JSONPath or XPath to the data root (e.g. `$.Rows` or `/Catalog/Book`).

Example:

```
CREATE CONNECTION remote_srv ON SFTP('sftp.example.com') WITH(USER='admin',
PASS='secret');
CREATE CONNECTION cloud_store ON AZUREBLOB('UseDevelopmentStorage=true')
WITH(CONTAINER='backup');
CREATE CONNECTION secure_db ON MSSQL('ENC:U2FsdGVkX1+...');
```

DROP CONNECTION Removes a previously defined connection from the current execution context.

Syntax: `DROP CONNECTION [IF EXISTS] <connection_name>;`

Example:

```
DROP CONNECTION IF EXISTS remote_srv;
```

USE DOCKER Spins up a containerized database instance (MsSql, Postgres, or Oracle) for temporary orchestration or testing.

Syntax: `USE DOCKER('<image_name>') [AS <alias>];`

Built-in Variables:

- `<alias>.CONNECTION_STRING`: Returns the dynamically generated connection string for the specified container.
- `DOCKER.CONNECTION_STRING`: Returns the connection string for the last started container (backward compatibility).

Management Commands:

- `<alias> CLOSE;`: Stops and disposes of the container.
- `<alias> STOP;`: Stops the running container (persists state).
- `<alias> START;`: Resumes a stopped container.
- `<alias> PAUSE;`: Pauses the container execution.
- `DOCKER CLOSE;`: Closes all active Docker containers.

Example:

```
-- Multiple containers with aliases
USE DOCKER('mcr.microsoft.com/mssql/server:2022-latest') AS dms;
DECLARE @conn varchar(500) = dms.CONNECTION_STRING;

USE DOCKER('postgres:15-alpine') AS dpost;
DECLARE @pconn varchar(500) = dpost.CONNECTION_STRING;

-- Management
dms STOP;
dms START;
dms CLOSE;
dpost CLOSE;
```

Variables & State Management

You have the ability to query from multiple different source databases and files. When using these different sources you have access to different processes and syntax. Knowing your context will help to avoid issues.

In the query below your context is the ETL-SQL engine.

```
CREATE CONNECTION c ON FLATFILE('C:\Users\chuck\scratch\ETL-
SQL\TestData\test_categories.csv');
DECLARE @date datetime = '01/01/2026';
SELECT @date;
```

But what if I wanted to run that on the database?

```
CREATE CONNECTION m ON MSSQL('localhost');

EXECUTE m
BEGIN
    DECLARE @date datetime = '01/01/2026';
    SELECT @date;
END
```

Now everything inside the the EXECUTE BEGIN .. .. END is run from the context of the Sql Server Engine.

What if I want the results

```
CREATE CONNECTION m ON MSSQL('localhost');

EXECUTE m INTO #emp
BEGIN
    SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > 1;
END
```

Now the ETL-SQL engine will run the query in the EXECUTE block and the results returned from that query will be stored in a temporary table called #emp in the ETL-SQL engine. So the results (#emp) now live in the context of ETL-SQL and as such it can transform, join, load them back into other connections.

In this scenario #emp is a temporary table in the ETL-SQL engine. It is not a table in the Sql Server database. It is a table in the ETL-SQL engine's memory.

What if I need to pass parameters down to the database?

```
CREATE CONNECTION m ON MSSQL('localhost');
DECLARE @id INT = 1;
```

```
       ,@name VARCHAR(50) = 'John';

EXECUTE m INTO #emp WITH(@id, @name)
BEGIN
    SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > ? AND t.[name] = ?;
END
```

This pushes the value of @id into the stored procedure as the first parameter. The ? is a placeholder for the parameter. You can add as many parameters as you need to. They will be processed in order.

What if I need to do way harder ones?

```
CREATE CONNECTION m ON MSSQL('localhost');
DECLARE @id INT = 1;
       ,@name varchar(50) = 'John';
       ,@stmt varchar(2000)
SET @stmt = 'SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > ' + @id + '
AND t.[name] = ''' + @name + ''';';

EXECUTE (
    @stmt
) AT m
```

Dynamic SQL is a powerful tool, but it can be dangerous. It is important to use it with caution.

My query is really straight forward. I just want to select from a table and get the results.

```
CREATE CONNECTION m ON MSSQL('localhost');

SELECT t.id, t.[name] INTO #temp FROM m.dbo.Employee AS t WHERE t.id > 1;
```

There is a shorthand you can use for simple select queries. You just need to add the connection to the table name in the FROM/JOIN clause. In the above example we have to assume it knows what database to connect to from either security or the connection string. The ETL-SQL engine will then run this query against the Sql Server connection and connect to the dbo.Employee table. It will then return the results to the ETL-SQL engine and store them in the #temp table.

Cool but the temp tables in the ETL-SQL engine do not have the correct data types.

```
CREATE CONNECTION m ON MSSQL('localhost');
CREATE TABLE #emp(
   id int
  ,name varchar(50)
);

INSERT INTO #emp (id, name)
SELECT t.id, t.[name] FROM m.dbo.Employee AS t WHERE t.id > 1;
```

```
-- OR
INSERT INTO #emp (id, name)
EXECUTE m
BEGIN
  SELECT t.id, t.[name] FROM dbo.Employee AS t WHERE t.id > 1;
END
```

You can explicitly define the columns you want to insert into the temp table. This is a good practice because it will prevent errors if the source table changes. You just have to make sure you have the same number of columns and the same data types or it will fail.

Querying & Filtering

`SELECT` Fetches, transforms, and projects data from an established connection or temporary memory table.

Supported clauses (Syntactic Order):

- `DISTINCT`: When used after `SELECT`, filters out duplicate rows from the final result set.
- `TOP <expression>`: Caps the number of returned rows (placed after `SELECT`). Supports literal numbers and variables.
- `INTO <target>`: (ETL specific) Streams the results directly into a destination connection or memory table (tables prefixed with `#`).
- `FROM <target>`: The source table or connection name. Supports aliases (e.g., `FROM my_conn AS T1`).
- `[INNER | LEFT | RIGHT | FULL | LEFT SEMI | LEFT ANTI] [HASH | LOOP | MERGE] JOIN <target> ON <condition>`: Combines data from multiple sources. You can optionally force a specific execution algorithm (`HASH`, `LOOP`, or `MERGE`) to explicitly optimize streaming performance against massive datasets.
  - `LEFT SEMI JOIN`: Returns rows from the left table where a match exists in the right table.
  - `LEFT ANTI JOIN`: Returns rows from the left table where *no* match exists in the right table.
- `[CROSS | OUTER] APPLY (<subquery>) <alias>`: Correlated subquery join. Allows the subquery to refer to columns from the left side of the apply.
- `WHERE <condition>`: Filters rows based on standard logical evaluations.
- `GROUP BY <columns>`: Aggregates datasets based on unique column pairings.
- `HAVING <condition>`: Filters result sets *after* GROUP BY aggregation has been applied.
- `PIVOT ( <aggregate_func>(<col>) FOR <pivot_col> IN (<values...>) ) AS <alias>`: Rotates a table-valued expression by turning unique values from one column in the expression into multiple columns in the output.
- `UNPIVOT ( <value_col> FOR <name_col> IN (<cols...>) ) AS <alias>`: Rotates a table-valued expression from a column-based form into a row-based form.
- `ORDER BY <column> [ASC|DESC] [, ...]`: Sorts the result set. Multiple columns are supported. `ASC` (default) or `DESC`. Can be used with `OFFSET`/`FETCH NEXT` for pagination.
- `OFFSET <expression> [ROWS]`: Skips a specific number of rows before returning results. Usually used with `ORDER BY`.
- `FETCH NEXT <expression> ROWS ONLY`: An alternative syntax for `LIMIT`, often used with `OFFSET`.
- `LIMIT <expression>`: Caps the number of returned rows (placed at the end of the query). Supports literal numbers and variables.

- `FOR JSON AUTO | PATH | RAW [, ROOT('name')] [, INCLUDE_NULL_VALUES] [, WITHOUT_ARRAY_WRAPPER]`: Formats results as a JSON string.
- `FOR XML AUTO | PATH | RAW [, ROOT('name')]`: Formats results as an XML string.

Example:

```
SELECT DISTINCT
    category,
    SUM(price) AS TotalSales,
    COUNT(DISTINCT transaction_id) AS UniqueItemsSold
INTO #sales_summary
FROM sales_db.transactions
WHERE created_at >= '2026-01-01'
GROUP BY category
HAVING TotalSales > @threshold
ORDER BY TotalSales DESC
LIMIT 10;
```

Logical Operators & Advanced Filters

Standard binary and logical operators are natively supported (`=`, `<`, `>`, `<=`, `>=`, `<>`, `AND`, `OR`).

`IN` and `NOT IN` Checks a field's value against an explicit list or a `LIST` variable.

```
WHERE category IN ('A', 'B')
   OR status NOT IN @validCats
```

`LIKE` Matches strings against SQL-standard wildcard patterns (`%` for any string, `_` for any single character).

```
WHERE email LIKE '%@gmail.com'
```

`EXISTS` and `NOT EXISTS` Evaluates whether a subquery returns any rows.

```
WHERE EXISTS (SELECT 1 FROM #temp WHERE id = main.id)
```

Set Operations

Combine results from multiple queries.

- `UNION ALL`: Combines all rows from both queries.
- `UNION`: Combines rows and removes duplicates.
- `EXCEPT`: Returns rows from the first query that are not in the second.
- `INTERSECT`: Returns rows present in both queries.

`CASE ... WHEN ... THEN ... ELSE ... END` Evaluates conditions sequentially to return specific projected outputs natively.

```
SELECT
    CASE
        WHEN id < 1000 THEN 'Legacy'
        WHEN id >= 1000 THEN 'Modern'
        ELSE 'Unknown'
    END AS SystemType
FROM system_table;
```

Data Manipulation Language (DML)

`INSERT INTO` Injects evaluated query outputs directly into a target table or connection.

Syntax: `INSERT INTO <connection_or_table> [(cols...)] [OUTPUT clauses...] SELECT <columns...> FROM <source> [WHERE <condition>];`

Example:

```
INSERT INTO sales_db.archive (category, TotalSales)
OUTPUT INSERTED.category, INSERTED.TotalSales INTO #AuditLog
SELECT category, TotalSales
FROM #sales_summary
WHERE TotalSales < 1000;
```

`BULK INSERT` Dramatically accelerates data loading directly by streaming payload binaries (such as CSVs) onto target databases or tables using strict destination schema adherence. Features advanced column-reordering maps natively.

Syntax: `BULK INSERT <connection_or_table> [(target_cols...)] FROM '<file_path>' WITH (FORMAT = 'CSV');`

Example:

```
BULK INSERT #Target (Name, Location, Age)
FROM 'TestData\test_bulk_mapped.csv'
WITH (FORMAT = 'CSV');
```

`UPDATE` Updates pre-existing datasets. Supports `OUTPUT` to capture before/after states via `DELETED` and `INSERTED` pseudo-tables.

Syntax: `UPDATE <connection.table> SET <col> = <val> [OUTPUT clauses...] [WHERE <condition>];`

Example:

```
UPDATE sales_db.archive
SET status = 'Closed'
OUTPUT DELETED.status AS OldStatus, INSERTED.status AS NewStatus
WHERE created_at < '2020-01-01';
```

`DELETE` Removes rows from a table.

Syntax: `DELETE FROM <connection.table> [OUTPUT clauses...] [WHERE <condition>];`

`TRUNCATE TABLE` Efficiently removes all rows from a table. Faster than `DELETE` for large datasets.

Syntax: `TRUNCATE TABLE <connection.table>;`

`CREATE TABLE` Defines a structure for storing data, including support for various constraints.

Syntax:

```
CREATE TABLE <table_name> (
    <col_name> <data_type> [IDENTITY] [PRIMARY KEY] [UNIQUE] [NOT NULL | NULL]
[DEFAULT <expr>] [CHECK (<expr>)] [REFERENCES <ref_table>(<ref_col>)],
    ...
    [CONSTRAINT <name>] PRIMARY KEY (<cols...>),
    [CONSTRAINT <name>] UNIQUE (<cols...>),
    [CONSTRAINT <name>] CHECK (<expr>),
    [CONSTRAINT <name>] FOREIGN KEY (<cols...>) REFERENCES <ref_table>
(<ref_cols...>)
);
```

Example:

```
CREATE TABLE Orders (
    OrderId INT IDENTITY PRIMARY KEY,
    OrderDate DATETIME DEFAULT GETDATE(),
    TotalAmount DECIMAL(18,2) NOT NULL CHECK (TotalAmount >= 0),
    CustomerId INT REFERENCES Customers(Id),
    Status VARCHAR(20) DEFAULT 'Pending'
);

-- Table-level multi-column constraint
CREATE TABLE OrderItems (
    OrderId INT,
    LineItem INT,
    PRIMARY KEY (OrderId, LineItem)
);
```

`ALTER TABLE` Modifies an existing table's structure.

Syntax:

- `ALTER TABLE <name> ADD <col_definition>;`
- `ALTER TABLE <name> DROP COLUMN <col_name>;`
- `ALTER TABLE <name> RENAME COLUMN <old_name> TO <new_name>;`

`CREATE INDEX` Creates an index on a table to improve query performance. *Syntax:* `CREATE [UNIQUE] INDEX <index_name> ON <table_name> (<column_name> [ASC|DESC] [, ...]);`

`DROP INDEX` Removes an existing index. *Syntax:* `DROP INDEX <table_name>.<index_name>;`

`MERGE` (UPSERT) Synchronizes data between a source and target by performing multiple DML actions (INSERT, UPDATE, DELETE) in a single statement. It is the core of incremental ETL processing.

Syntax:

```
MERGE INTO <target> [AS T]
USING <source_table_or_subquery> [AS S]
ON <match_condition>
[WHEN MATCHED [AND <condition>] THEN <action>]
[WHEN NOT MATCHED [BY TARGET] [AND <condition>] THEN <action>]
[WHEN NOT MATCHED BY SOURCE [AND <condition>] THEN <action>];
```

Supported Actions:

- `UPDATE SET col = val, ...`
- `INSERT (cols...) VALUES (vals...)`
- `DELETE`

Example:

```
MERGE INTO target_table AS T
USING (SELECT * FROM staging_table) AS S
ON T.id = S.id
WHEN MATCHED THEN
    UPDATE SET name = S.name, updated_at = GETDATE()
WHEN NOT MATCHED THEN
    INSERT (id, name) VALUES (S.id, S.name)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;
```

Statements

DDL & Resource Management

CREATE TABLE

Creates a new table or virtual view.

```
CREATE TABLE table_name AS SELECT ...;
```

DROP TABLE

Deletes a table from the engine's memory or the target data source.

```
DROP TABLE table_name;
```

DROP FUNCTION / PROCEDURE

Removes a user-defined function or procedure.

```
DROP FUNCTION MyFunc;
DROP PROCEDURE MyProc;
```

Control Flow

IF...ELSE

Conditional execution.

```
IF @val > 10
BEGIN
    PRINT 'High';
END
ELSE
BEGIN
    PRINT 'Low';
END
```

WHILE

Loop execution.

```
WHILE @i < 10
BEGIN
    SET @i = @i + 1;
    IF @i = 5 CONTINUE;
    IF @i = 8 BREAK;
END
```

BREAK / CONTINUE

Control the flow of a `WHILE` loop. `BREAK` exits the loop immediately, and `CONTINUE` skips to the next iteration.

TRY...CATCH

Error handling block.

```
BEGIN TRY
    -- potentially failing code
END TRY
BEGIN CATCH
    PRINT 'Error: ' + ERROR_MESSAGE();
END CATCH
```

RAISERROR / THROW

Manually raise an error.

```
RAISERROR('Custom error message', 16, 1);
-- or
THROW 50000, 'Error message', 1;
```

WAITFOR

Pause execution for a specific duration or until a specific time.

```
WAITFOR DELAY '00:00:05'; -- Wait 5 seconds
WAITFOR TIME '23:59:59';  -- Wait until midnight
```

Variable Management

DECLARE

Defines a variable with a specific type.

```
DECLARE @name STRING = 'Chuck';
DECLARE @list LIST = [1, 2, 3];
```

SET

Assigns a value to an existing variable.

```
SET @name = 'Charles';
```

Data Movement & Transformation

EXECUTE / EXEC

Executes a stored procedure or a dynamic SQL block.

```
EXECUTE RemoteProc @param1 = 'val';
EXEC('SELECT * FROM Table') AT RemoteConn;
```

INSERT INTO

Appends data to a table.

```
INSERT INTO TargetTable SELECT * FROM SourceTable;
```

TRUNCATE TABLE

Removes all rows from a table without dropping its schema.

```
TRUNCATE TABLE TempStaging;
```

Job & Profile Management

SHOW JOBS

Lists all currently running or recently completed background jobs.

```
SHOW JOBS;
```

KILL JOB

Terminates a running background job by its ID.

```
KILL JOB 'job_id_123';
```

SET PROFILING

Enables or disables performance profiling for the session.

```
SET PROFILING ON;
-- run scripts
SET PROFILING OFF;
```

SHOW PROFILE

Displays the timing and resource usage for the most recently executed statements.

```
SHOW PROFILE;
```

Remote File Management

GET_FILE / PUT_FILE

Transfers files between the local system and a remote connector (SFTP, FTP, Azure Blob).

```
GET_FILE 'remote/path/data.csv' TO 'local/data.csv' FROM @MySftp;
PUT_FILE 'local/upload.txt' TO 'uploads/upload.txt' AT @AzureBlob;
```

- `LEFT(string, n)`, `RIGHT(string, n)`: Extracts `n` characters from either side.
- `CHARINDEX(substring, string)` (or `INSTR(string, substring)`): Finds the 1-based index position of a substring. Not: `CHARINDEX` takes `(sub, str)` while `INSTR` takes `(str, sub)`.
- `REVERSE(string)`: Reverses the string characters.
- `COALESCE(val1, val2...)`: Returns the first non-null value in a list.
- `ISNULL(val1, val2)`: Returns `val2` if `val1` is null.
- `NULLIF(val1, val2)`: Returns NULL if the two values are equal, otherwise returns the first value.
- `CAST(expr AS type)`: Converts an expression to a specific type (`INT`, `STRING`, `DECIMAL`, `DATE`).
- `TRY_CAST(expr AS type)`: Similar to `CAST`, but returns `NULL` if the conversion fails instead of throwing an error.
- `STUFF(string, start, length, new_string)`: Deletes a specified length of characters and inserts a new sequence of characters at a specified starting point.
- `STRING_ESCAPE(text, type)`: Escapes special characters in a string based on the specified type (e.g., `'json'`).
- `STRING_SPLIT(string, separator)`: Splits a string into a list of substrings based on a specified separator.
- `ASCII(string)`, `UNICODE(string)`: Returns the integer ASCII or Unicode code point of the first character of the input string.
- `CHAR(int)`: Converts an integer ASCII code to its character equivalent.
- `FORMAT(value, format)`: Formats a value (Date or Numeric) based on a .NET format string.
- `PATINDEX(pattern, string)`: Returns the starting position of the first occurrence of a pattern in a specified expression. Supports `%` and `_` wildcards.

- `STR(float, [length], [decimal])`: Returns character data converted from numeric data. The `length` is the total length (including decimal point and sign), and `decimal` is the number of places to the right of the decimal point.
- `QUOTENAME(string, [quote_char])`: Returns a Unicode string with the delimiters added to make the input string a valid SQL Server delimited identifier. Default is `[]`.
- `TRANSLATE(string, from, to)`: Returns the string provided as a first argument after some characters specified in the second argument are translated into a destination set of characters specified in the third argument.
- `DATALENGTH(value)`: Returns the number of bytes used to represent any expression.
- `TO_STR(value)`: A convenience alias for `CAST(value AS STRING)`.
- `REPLICATE(string, count)`: Repeats a string value a specified number of times.

List Scalars

- `LENGTH(list)`: Returns the number of items in a list.
- `SORT_LIST(list[, 'ASC'|'DESC'])`: Returns a sorted version of the list.
- `APPEND_TO_LIST(@list, value)`: Adds an item to a list variable.
- `REMOVE_FROM_LIST(@list, value)`: Removes an item from a list variable.
- `LPAD(str, len[, pad])`, `RPAD(str, len[, pad])`: Pads a string to a specific length.
- `INITCAP(str)`: Capitalizes the first letter of each word.
- `POSITION(sub IN str)` or `STRPOS(str, sub)`: Returns the 1-based index of a substring.
- `LEAST(v1, v2...)`, `GREATEST(v1, v2...)`: Returns the smallest or largest value from a list.
- `DECODE(val, search1, result1, [search2, result2, ...], default)`: Oracle-style CASE shorthand.

Math Scalars

- `ROUND(numeric, length)`: Rounds to the specified decimal precision.
- `ABS(numeric)`: Absolute value.
- `CEILING(numeric)`, `FLOOR(numeric)`: Rounds up or down to the nearest integer.
- `POWER(base, exp)`: Exponential calculation.
- `SQRT(float)`: Square root.
- `RAND([seed])`: Returns a random float between 0 and 1.
- `SIN(float)`, `COS(float)`, `TAN(float)`: Standard trigonometric functions (input in radians).
- `ASIN(float)`, `ACOS(float)`, `ATAN(float)`: Inverse trigonometric functions (returns radians).
- `ATAN2(y, x)`: Returns the angle in radians between the positive x-axis and the point (x, y).
- `SIGN(numeric)`: Returns the sign of a number: `1` for positive, `-1` for negative, and `0` for zero.

Date and Time Scalars

- `CURRENT_TIMESTAMP`, `GETDATE()`, `GETUTCDATE()`: Functions returning current system time.
- `DATEADD(datepart, number, date)`: Adds a specific interval to a date.
- `DATEDIFF(datepart, startdate, enddate)`: Returns the difference between two dates.
- `DATEPART(datepart, date)`, `DATENAME(datepart, date)`: Extracts parts of a date.
- `YEAR(date)`, `MONTH(date)`, `DAY(date)`: Extract integer components from a date.
- `EOMONTH(date[, months_to_add])`: Returns the last day of the month containing the specified date.
- `ISDATE(expr)`: Validates if an expression can be converted to a valid date.
- `DATEFROMPARTS(year, month, day)`: Constructs a new date.

- `DATETIMEFROMPARTS(year, month, day, hour, minute, second, ms)`: Constructs a new datetime.
- `TIMEFROMPARTS(hour, minute, second, fractions, precision)`: Constructs a new time.
- `DATETIMEOFFSETSFROMPARTS(...)`: Constructs a datetime with offset.
- `FORMAT(value, format)`: Formats an object based on a C# `ToString` configuration string.
- `TRUNC(date)`, `TO_DATE(date)`: Truncates time portions from a datetime.
- `expr AT TIME ZONE 'timezone_id'`: Converts a datetime to the specified timezone. Default input kind is UTC if not specified.

Cryptography & Hashing

- `HASHBYTES('algorithm', value)`: Returns a hash of the input using `MD5`, `SHA1`, `SHA2_256`, or `SHA2_512`.
- `CHECKSUM(val1, val2...)`: Returns a high-uniqueness numeric hash (based on truncated SHA-256) for the combined inputs.
- `BINARY_CHECKSUM(val1, val2...)`: Returns a binary-compatible hash.
- `NEWID()`: Returns a new unique identifier (GUID).
- `NEWSEQUENTIALID()`: Returns a new GUID optimized for sequential insertion.

Aggregation

- `COUNT([DISTINCT] col)`: Aggregation tally. If `DISTINCT` is specified, only unique non-null values are counted.
- `SUM(col)`: Aggregation total.
- `MIN(col)`: Aggregation smallest value.
- `MAX(col)`: Aggregation highest value.
- `AVG(col)`: Aggregation mathematical mean.
- `STRING_AGG(col, separator) [WITHIN GROUP (ORDER BY col [ASC|DESC])]`: Concatenates values from multiple rows into a single string, separated by the specified string. Optionally orders the values before concatenation. NULL values are ignored.

Windows Functions

Window functions operate on a set of rows and return a single value for each row from the underlying query. The `OVER` clause defines the window or user-specified set of rows.

`LAG(col[, offset[, default]])` **and** `LEAD(col[, offset[, default]])` Accesses data from a previous or subsequent row in the same result set without the use of a self-join.

`FIRST_VALUE(col)` **and** `LAST_VALUE(col)` Returns the first or last value in an ordered set of values.

`NTILE(n)` Distributes rows into `n` specified number of ranked groups (buckets). Rows are distributed as evenly as possible.

Window Framing (`ROWS` | `RANGE`) Window functions support framing to restrict the rows within the partition used for calculation.

- `ROWS BETWEEN <start> AND <end>`: Specifies a physical offset from the current row.
- `RANGE BETWEEN <start> AND <end>`: Specifies a logical range based on values.

Bounds:

- `UNBOUNDED PRECEDING` / `FOLLOWING`
- `<n> PRECEDING` / `FOLLOWING`
- `CURRENT ROW`

```
SELECT
    id,
    amount,
    SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
AS running_total,
    AVG(amount) OVER(ORDER BY id ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS
moving_avg
FROM #sales;
```

Loops and Flow Control

Procedural loops are supported natively via C# execution within the ETL-SQL evaluator to handle repetition.

`BEGIN ... END` A Block wrapper that allows for multiple statements inside flow controls.

`IF ... ELSE IF ... ELSE` Branches data operations conditionally.

```
IF @amount > 1000
BEGIN
    INSERT INTO #high_value SELECT * FROM #sales WHERE amount > 1000;
END
ELSE IF @amount > 500
BEGIN
    INSERT INTO #mid_value SELECT * FROM #sales WHERE amount > 500;
END
ELSE
BEGIN
    INSERT INTO #low_value SELECT * FROM #sales;
END;
```

`WHILE` Repeats a statement or block while a specified condition is true.

```
DECLARE @accumulator INT = 0;
WHILE @accumulator < 5
BEGIN
SET @accumulator = @accumulator + 1;
END;
```

`EXECUTE` **(Remote Execution)** Executes SQL code on a remote connection. Supports capturing results into local tables and passing parameters for secure, parameterized execution.

Connection Block Form:

```
EXECUTE ds [INTO #target] [WITH (@param1, @param2, ...)]
BEGIN
  -- Remote SQL dialect (e.g. T-SQL for MSSQL)
  SELECT id, name FROM remote_table WHERE category_id = ?;
END;
```

String Literal Form:

```
EXECUTE (
  'SELECT id, name FROM remote_table WHERE category_id = ?'
) AT ds [INTO #target] [WITH (@cat_id)];
```

Key Parameters:

- **INTO #table**: Streams results from the remote execution directly into a local ETL-SQL memory table.
- **WITH (@vars)**: Passes local variables to the remote server. Use standard `?` placeholders in the remote SQL script; variables are applied in the order they are listed.
- **Note**: The string literal form sends raw SQL directly to the server, so it requires valid target server syntax and standard single-quote escaping (`''`).

`PARALLEL` The `PARALLEL` keyword allows for concurrent execution of multiple statements or blocks. This is particularly useful for independent data streams that do not have inter-dependencies, such as loading multiple dimension tables simultaneously.

Syntax:

```
PARALLEL
BEGIN
    <statement_1>;
    <statement_2>;
    ...
END
```

Key Characteristics:

- **Non-blocking**: Statements within a `PARALLEL` block are fired concurrently.
- **Wait-all**: The execution engine waits for *all* statements in the block to complete before moving to the next statement outside the block.
- **Isolated Scopes**: Each branch typically operates on its own connection state to avoid race conditions, though global variables are accessible.

Example:

```
-- Load three independent tables in parallel
PARALLEL
BEGIN
    SELECT * INTO #Dimensions_Date FROM src_db.DateDim;
    SELECT * INTO #Dimensions_Product FROM src_db.ProductDim;
    SELECT * INTO #Dimensions_Store FROM src_db.StoreDim;
END

-- Sequential execution resumes here after all three above are finished
```

`FOR` Iterates a variable through a numeric range with an optional `STEP`.

```
FOR @idx = 100 TO 95 STEP -1
BEGIN
    INSERT INTO #results (loop_val) VALUES (@idx);
END;
```

`FOREACH` Iterates comprehensively through a designated `LIST` variable.

```
DECLARE @result_list LIST = [10, 20, 30];
FOREACH @val IN @result_list
BEGIN
    INSERT INTO #results (loop_val) VALUES (@val);
END;
```

`RUN SCRIPT` Executes another ETL-SQL script file, optionally passing parameters. *Syntax:* `RUN SCRIPT '<script_path>' [WITH (@param1 = val1, ...)];`

Example:

```
RUN SCRIPT 'sub_process.etlsql' WITH (@batchId = 1234, @env = 'PROD');
```

9. Modular ETL (Procedures & Functions)

`CREATE PROCEDURE` Defines a reusable block of ETL-SQL statements.

```
CREATE PROCEDURE ArchiveSales @olderThan DATE
AS
BEGIN
    INSERT INTO archive_db.sales SELECT * FROM sales_db.sales WHERE created_at <
@olderThan;
    DELETE FROM sales_db.sales WHERE created_at < @olderThan;
END;
```

```
EXEC ArchiveSales '2025-01-01';
```

`CREATE FUNCTION` Defines a User-Defined Function (UDF) that returns a scalar value.

```
CREATE FUNCTION CalculateTax(@amount DECIMAL) RETURNS DECIMAL
AS
BEGIN
    RETURN @amount * 0.15;
END;

SELECT id, CalculateTax(price) AS tax FROM #sales;
```

10. Transactions & Error Handling

Transactions Supports atomic operations via a transaction stack.

- `BEGIN TRANSACTION` (or `BEGIN TRAN`)
- `COMMIT` (or `COMMIT TRAN`)
- `ROLLBACK` (or `ROLLBACK TRAN`)
- `@@TRANCOUNT`: Built-in variable returning the current nesting level.

Error Handling

- `TRY...CATCH`: Standard block for capturing runtime exceptions.
- `THROW [number, 'message', state]`: Raises a custom error.

```
BEGIN TRY
    BEGIN TRANSACTION;
    -- Operations...
    IF @errorCondition = 1 THROW 50000, 'Custom Error', 1;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT('Error: ' + ERROR_MESSAGE());
END CATCH;
```

11. Automation & Introspection

File & Directory Operations Specialized commands for filesystem management. Supports **Connection-based Path Resolution** (e.g., `MyDir + '/file.csv'` where `MyDir` is a connection name).

- `COPY_FILE('src', 'dest')`
- `MOVE_FILE('src', 'dest')`
- `RENAME_FILE('src', 'new_name')`
- `DELETE_FILE('path')`

- `COMPRESS_FILE('src'[, 'dest'])`
- `ENCRYPT_FILE('src'[, 'dest'])` (Uses master password)
- `DECRYPT_FILE('src'[, 'dest'])`

Directory Management

- `CREATE_DIRECTORY('path')`
- `DELETE_DIRECTORY('path')`
- `RENAME_DIRECTORY('path', 'new_name')`
- `MOVE_DIRECTORY('path', 'dest')`
- `COPY_DIRECTORY('src', 'dest')`
- `DELETE_DIRECTORY_CONTENTS('path')`

Docker Operations

- `START_DOCKER <alias>`: Starts a Docker container for the specified connection.
- `STOP_DOCKER <alias>`: Stops a running Docker container.
- `PAUSE_DOCKER <alias>`: Pauses a running Docker container.
- `CLOSE_DOCKER <alias>`: Stops and removes a Docker container.

Utility Commands

- `EXPLAIN <query>`: Displays the query execution plan.
- `HELP CONNECTION <type>`: Displays help and options for a specific connection provider.
- `PRINT(message[, timestamp[, format]])`: Outputs a message to the console.

Static Analysis & Linting

- `LINT '<script_path>';`: Analyzes the specified script for errors and best practices (e.g., missing `WHERE` clauses on `DELETE`, undeclared variables). Returns a table of findings.

Job Scheduling & History

- `CREATE JOB <job_name> ON <schedule_cron> AS '<script_path>';`: Schedules a script to run automatically.
- `SHOW JOB HISTORY [<job_name>];`: Displays the execution history and status of scheduled jobs.

`LINEAGE` The lineage system provides end-to-end traceability of data movement, capturing every transformation, source-to-target mapping, and metadata tag inheritance.

Syntax:

- `LINEAGE(<target_table> [, <column_name>]) [TO '<file_path>'];`
- `SELECT * FROM LINEAGE(<target_table> [, <column_name>]);`

Variants:

1. **Console View**: `LINEAGE(#FinalAudit);` displays a hierarchical tree of sources in the console output.
2. **Column Trace**: `LINEAGE(#FinalAudit, 'UserId');` filters the trace to a single column's ancestry.
3. **Markdown Export**: `LINEAGE(#FinalAudit) TO 'reports/lineage.md';` generates a comprehensive Markdown report including a **Mermaid.js** relationship diagram and a detailed audit table.

4. **Queryable Source**: `SELECT Operation, SourceTables FROM LINEAGE(#FinalAudit);` allows you to treat the lineage audit log as a standard table for automated validation or reporting.

Queryable Columns:

- `Timestamp`, `Operation`, `TargetTable`, `TargetColumn`, `SourceTables`, `SourceColumns`, `Description`, `Metadata` (JSON), `DerivedFromDescriptions`, `SourceFile`, `Line`, `Column`.

Metadata Inheritance & Amalgamation

When columns are combined or transformed (e.g., `UnitPrice * Qty AS Total`), the system automatically propagates metadata:

1. **Last-Seen Wins**: The primary description (`@d`) and individual custom tags are inherited from the last source column in the expression that has them defined.
2. **Amalgamation**: The `DerivedFromDescriptions` field is populated with a structured list of all involved source descriptions (e.g., `UnitPrice: Base price per unit, Qty: Number of items sold`), ensuring no context is lost during transformations.
3. **Global Persistence**: All tags assigned anywhere in the lineage chain are preserved and queryable at the final destination.

Lineage Transformation Functions

- `GET_TAGS(table_name [, column_name])`: Returns a `LIST` of all custom metadata tag names defined for a table or specific column.
- `GET_TAG_VALUE(table_name, column_name, tag_name)`: Returns the string value of a specific metadata tag.

Example:

```
DECLARE @tags LIST = GET_TAGS('#TaggedUsers', 'UserId');
IF 'sensitive' IN @tags
BEGIN
    PRINT('Warning: Table contains sensitive data.');
END;
```

Attach arbitrary metadata tags to columns and tables using special comment blocks. Tags are captured by the lineage system for auditing, governance, and UI documentation. `@d:` is the reserved description tag.

Column Tags — placed immediately after a column expression in a `SELECT` list:

```
col_name /* @d: Description text; @tag: value; @another: value2; */
```

Table Tags — placed immediately after a table reference in `FROM` or `JOIN`:

```
FROM table_name /* @owner: TeamA; @sensitivity: high; */
```

Example:

```
SELECT
    UserId  /* @d: Internal user ID; @sensitive: true; */,
    UserName /* @d: Full name of the user; @owner: Chuck; */
INTO #TaggedUsers
FROM m.Users /* @owner: SecurityTeam; */;
```

Email Operations

- `SEND_EMAIL TO '<to>' SUBJECT '<subject>' BODY '<body>' [AT <connection>];`: Sends an automated email alert (requires an SMTP connection).